


```

d_a = bytes_to_long(os.urandom(32))
P_a = G * d_a

printable = [ord(char.encode()) for char in string.printable]

def encrypt(msg : bytes, pubkey : ECPoint):
    x = bytes_to_long(msg)
    y = modular_sqrt(x**3 + a*x + b, p)
    m = ECPoint(curve, x, y)
    d_b = number.getRandomRange(0,n)
    return (G * d_b, m + (pubkey * d_b))

def decrypt(B : ECPoint, c : ECPoint, d_a : int):
    if B.inf or c.inf: return b''
    return long_to_bytes((c - (B * d_a)).x)

def loop():
    print('I will decrypt anything as long as it does not talk about flags.')
    balance = 1024
    while True:
        print('B:', end = '')
        sys.stdout.flush()
        B_input = sys.stdin.buffer.readline().strip().decode()
        print('c:', end = '')
        sys.stdout.flush()
        c_input = sys.stdin.buffer.readline().strip().decode()
        B = ECPoint(curve, *[int(_) for _ in B_input.split(',')])
        c = ECPoint(curve, *[int(_) for _ in c_input.split(',')])
        msg = decrypt(B, c, d_a)
        if b'ENO' in msg:
            balance = -1
        else:
            balance -= 1 + len([c for c in msg if c in printable])
        if balance >= 0:
            print(hexlify(msg))
            print('balance left: %d' % balance)
        else:
            print('You cannot afford any more decryptions.')
            return

if __name__ == '__main__':
    print('My public key is:')
    print(P_a)
    print('Good luck decrypting this cipher.')
    B,c = encrypt(flag1, P_a)
    print(B)
    print(c)
    key = long_to_bytes((d_a >> (8*16)) ^ (d_a & 0xffffffffffffffffffffffffffffffff))
    enc = AES.new(key, AES.MODE_ECB)
    cipher = enc.encrypt(flag2)
    print(hexlify(cipher).decode())
    try:
        loop()
    except Exception as err:
        print(repr(err))

```

Curvy Decryptor part 1

The solution of Curvy Decryptor part 1 is to find out flag1

Curvy Decryptor part 2

The solution of Curvy Decryptor part 1 is to find out flag2

The flag2 appears to be AES encrypted with a key which is

```
key = long_to_bytes((d_a >> (8*16)) ^ (d_a & 0xffffffffffffffffffffffffffffffff))
```

The xor of most significant and least significant 64 bits of the 128 bits private key `d_a`
So in order to recover the `flag2` we will need to break the ECC and recover `d_a`

Analysing curvy_decryptor.py

[illegible]

indeed look like a standard curve P-256 (note that $a = -3$ is equivalent to $a = p - 3$)
 So there doesn't appear to be any standard weakness based on weak curve parameters.
 The private key d_a is initialized to be per-instance system-random 128 bit number
 And the Public key P_a is simply $G * d_a$

Encryption

```
def encrypt(msg : bytes, pubkey : ECPoint):
    x = bytes_to_long(msg)
    y = modular_sqrt(x**3 + a*x + b, p)
    m = ECPoint(curve, x, y)
    d_b = number.getRandomRange(0,n)
    return (G * d_b, m + (pubkey * d_b))
```

The msg is encoded as the x coordinate of the point, the corresponding y is found so as to find the point on the curve to generate the point m

A secure random number d_b in range $(0 - \text{curve order})$ is generated as the nonce, The points $G \cdot d_b$ and $m + (\text{pubkey} \cdot d_b)$ are returned

Decryption

```
def decrypt(B : ECPPoint, c : ECPPoint, d_a : int):  
    if B.inf or c.inf: return b''  
    return long_to_bytes((c - (B * d_a)).x)
```

It simply reverses the encrypt function if correct d_a is provided
i.e if we do

```
B,c = encrypt(flag1, P_a)  
xx = decrypt(B,c)
```

We will get,

$$\begin{aligned} & \text{decrypt}((G * d_b * d_a), \text{flag}_m + (P_a * d_b)) \\ &= (\text{flag}_m + (P_a * d_b) - G * d_b * d_a).x \\ &= (\text{flag}_m + G * d_a * d_b - G * d_b * d_a).x \\ &= (\text{flag}_m).x = \text{flag} \end{aligned}$$

Main Loop

The main loop of the program just repeatedly asks for input of two EC points B and c and tries to decrypt it with the servers private key d_a

BUT

if the decryption contains `b'ENO'` i.e the start of the flag, it exits.

Otherwise it decreases the balance proportionate to the number of printable characters in the decryption.

So If we directly input the B and c corresponding to the `flag1`, it will abort and hence no flags for us :(

But it doesnt care what points we ask it to decrypt.

So what if we try to decrypt the points B, $c + A$

We will get,

$$\begin{aligned} & \text{decrypt}((G * d_b * d_a), \text{flag}_m + (P_a * d_b) + A) \\ &= (\text{flag}_m + (P_a * d_b) + A - G * d_b * d_a).x \\ &= (\text{flag}_m + A + G * d_a * d_b - G * d_b * d_a).x \\ &= (\text{flag}_m + A).x \end{aligned}$$

As long as we know A, we can always get the point from the x coordinate, and subtract A from it to get the original point from the curve for the sake of simplicity, we can even pick it to be G

or we can even try decrypting the points $B + A$, c which will lead to

$$\begin{aligned} & \text{decrypt}((G * d_b * d_a) + A, \text{flag}_m + (P_a * d_b)) \\ &= (\text{flag}_m + (P_a * d_b) - (G * d_b + A) * d_a).x \\ &= (\text{flag}_m + G * d_a * d_b - G * d_b * d_a - A * d_a).x \\ &= (\text{flag}_m - A * d_a).x \end{aligned}$$

if we choose A to be G or -G, we will end up with the point flag - P_a or flag + P_a and since we even know P_a, it will also work

With a high probability, we wont observe any **b'ENO'** in the resulting point, and if we do, we can always pick countless possibilities of **A** to make it work

```
from ec import *
from utils import *
import pwn

p = 0xffffffff000000010000000000000000000000fffffffffffffffffffffffa
a = -3
b = 0x5ac635d8aa3a93e7b3ebbd55769886bc651d06b0cc53b0f63bce3c3e27d2604b
n = 0xffffffff00000000fffffffffffffbce6faada7179e84f3b9cac2fc632551
curve = EllipticCurve(p,a,b, order = n)
G = ECPoint(curve, 0x6b17d1f2e12c4247f8bce6e563a440f277037d812deb33a0f4a13945d898c296,
0x4fe342e2fe1a7f9b8ee7eb4a7c0f9e162bce33576b315ececb6406837bf51f5))

HOST, PORT = "52.59.124.14", 10005
REM = pwn.remote(HOST, PORT)

REM.recvline() # My public key is:
pubkey = REM.recvline().strip()[6:-1].split(b',')
P_a = ECPoint(curve, int(pubkey[0]), int(pubkey[1]))

REM.recvline() # Good luck decrypting this cipher.
B_text = REM.recvline().strip()[6:-1].split(b',')
B = ECPoint(curve, int(B_text[0]), int(B_text[1]))

c_text = REM.recvline().strip()[6:-1].split(b',')
c = ECPoint(curve, int(c_text[0]), int(c_text[1]))

flag2_enc = bytes.fromhex(REM.recvline().strip().decode())

REM.recvline() # I will decrypt anything as long as it does not talk about flags.

def get_decryption(B,c):
    REM.sendline("{}{}".format(B.x, B.y))
    REM.sendline("{}{}".format(c.x, c.y))
    status = REM.recvline()
    if b'cannot afford' in status:
```

```

        return -1, None
    balance = int(REM.recvline().strip().split(b': ')[-1])
    return balance, bytes.fromhex(status.strip()[6:-1].decode())

bal, BG = get_decryption(B, c+G)
BG_int = int.from_bytes(BG)
y = modular_sqrt(BG_int**3+a*BG_int+b,p) # getting the valid y coordinate for the x
point_BG1 = ECPoint(curve, BG_int, y)
point_BG2 = -point_BG1
print(int.to_bytes((point_BG1-G).x, 32,'big'))
print(int.to_bytes((point_BG2-G).x, 32,'big'))

```

Note that we only get the x coordinate of the given point lifting the point would result in two points and we will need to try with both of them (x,y) and (x,-y)

The last part will also work with

```

bal, BG = get_decryption(B-G, c)
BG_int = int.from_bytes(BG)
y = modular_sqrt(BG_int**3+a*BG_int+b,p) # getting the valid y coordinate for the x
point_BG1 = ECPoint(curve, BG_int, y)
point_BG2 = -point_BG1
print(int.to_bytes((point_BG1-P_a).x, 32,'big'))
print(int.to_bytes((point_BG2-P_a).x, 32,'big'))

```

And we get the flag to part 1 b'\x00\x00EN0{ElGam4l_1s_multipllcativ3}'

Analysing ec.py

```

from Crypto.Util.number import inverse

class EllipticCurve(object):
    def __init__(self, p, a, b, order = None):
        self.p = p
        self.a = a
        self.b = b
        self.n = order

    def __str__(self):
        return 'y^2 = x^3 + %dx + %d modulo %d' % (self.a, self.b, self.p)

    def __eq__(self, other):
        return (self.a, self.b, self.p) == (other.a, other.b, other.p)

class ECPoint(object):
    def __init__(self, curve, x, y, inf = False):
        self.x = x % curve.p
        self.y = y % curve.p
        self.curve = curve
        self.inf = inf
        if x == 0 and y == 0: self.inf = True

    def copy(self):
        return ECPoint(self.curve, self.x, self.y)

    def __neg__(self):

```

```

        return ECPoint(self.curve, self.x, -self.y, self.inf)

def __add__(self, point):
    p = self.curve.p
    if self.inf:
        return point.copy()
    if point.inf:
        return self.copy()
    if self.x == point.x and (self.y + point.y) % p == 0:
        return ECPoint(self.curve, 0, 0, True)
    if self.x == point.x:
        lamb = (3*self.x**2 + self.curve.a) * inverse(2 * self.y, p) %
p
    else:
        lamb = (point.y - self.y) * inverse(point.x - self.x, p) % p
    x = (lamb**2 - self.x - point.x) % p
    y = (lamb * (self.x - x) - self.y) % p
    return ECPoint(self.curve,x,y)

def __sub__(self, point):
    return self + (-point)

def __str__(self):
    if self.inf: return 'Point(inf)'
    return 'Point(%d, %d)' % (self.x, self.y)

def __mul__(self, k):
    k = int(k)
    base = self.copy()
    res = ECPoint(self.curve, 0,0,True)
    while k > 0:
        if k & 1:
            res = res + base
        base = base + base
        k >>= 1
    return res

def __eq__(self, point):
    return (self.inf and point.inf) or (self.x == point.x and self.y ==
point.y)

```

Looking closely at the ECPoint class, one would note that on the initialization of the point with arbitrary x , y coordinates, it works as usual and doesn't check whether the supplied x , y satisfy the curve equation $y^2 = x^3 + ax + b \pmod p$

This leads to an interesting vulnerability aka **Invalid Curve Attack**

Which can be noted by the facts that

1. The point addition of two points P and Q over the curve $y^2 = x^3 + ax + b \pmod p$ if $P \neq Q$ is independent of both the curve parameters a and b
2. The point doubling i.e $P = Q$ is just dependent on P and Q and a but not on b again

This means that the group addition operation is independent of the parameter b , the number of points in the group of some point P is just dependent on P and a but independent of b

So if we choose a curve C' with parameter b' and pick a valid point P' on it, and run the point addition over the original curve C , the order of point P' when used on C will be the same as order of point P' when used on C'

This implies that we are not stuck with the original prime order of P-256, but we can vary $b = -3$ such that the order of the curve C' has small factors. We can then easily find points P' with those small factors as their order by using the fact that -

If G' is the generator of C' with order $o = f_1 f_2 f_3 \dots f_n$, the point $G' * (o/f_1)$ will have the order f_1

Once we have sufficient number of small orders, we can take a chinese remainder theorem over them to recover the original private key d_a

To find the order of curve C' and the corresponding generators, we can utilize the greater library of sagemath

```
def get_invalid_curves(a,b,n,cutoff=10**5):
    factors, total, i = {}, 0, 0
    while total < n*2:
        i += 1
        try:
            E = EllipticCurve(GF(p), [a,i])
            order = E.order()
            n_facs = order.factor()
        except ArithmeticError: #the parameter i defines a singular curve
            continue
        for prime, power in n_facs:
            if prime > cutoff: # dont take any factors bigger than it
                break
            gen = E.gen(0)*(order//prime)
            factors[prime] = [int(gen[0]), int(gen[1]),i]
            total *= prime
        print(i, total)
    # key : [gen_x, gen_y, b']
    return factors
```

And we can easily bruteforce a given point

```
def bruteforce(point, generator, order):
    for i in tqdm(range(order),desc=f"bruteforcing {order=}"):
        if point.y == (generator*i).y:
            return i
```

We get the following invalid curves and we just searched over 12 values of b !

```

invalid_curves = {
3: [79692280239272980873245387831874823476097665365069163558817570386218657526967,
15885657487155030912288031888128427124936813080859472376494663130798867119982, 12],
5: [30463586456259052716174121724723788478797318939762291523651966151233767925799,
14521026652335616630611219515390291411023400641948957680811406721043438902186, 12],
7: [7494160963166719022445789448670075468300539216220596044581361034311676798234,
98511591492536111584332615786483658886888663084020823615343609813895225923287, 0],
13: [44238399751822344629155927349410921734336660036385908812849527496419061724190,
111209137730801733774021088162408683888753865848469665403753663663899601005809, 3],
17: [9274144687945784364291903707116312659963917031850121885976057784793297477861,
86301980488975426887521079169756244075123784201450393961147458432303011672794, 10],
19: [108657251488837839710095894743866739052486880271258033613510419634191398226376,
8593905934316229092193387452437731577526088690676465668457131094758391852209, 4],
37: [6829338390266237482283310246665103308891228336319477318479644522260556056309,
74701668267551028200304338837410580676774474001240882947774298643661074111881, 3],
71: [47689891150662520216418276050802771367044708366511766907543187521601344242001,
76986723505258444611874235435887405018513758524921757765033540214285112109625, 1],
97: [109352438132789597676269849271161933029115963700376783044214805643475162939438,
40494199582133551395560104592591896448854412368997470369685720658455096277720, 3],
113: [24758423058742208238204864443231318968571918830166822957638906079202832915346,
64680694216633390865014362020707904150333340051904806580803858267985332824228, 3],
179: [83115631016490504822328655777895864162660782325660359674792065332260812135544,
3570714148691635381635812390035688673488260709581875628819622187261352405569, 4],
251: [22589597796365257246296758128505770638799961769310824687868041010311103597978,
22730375842099404560129412560882492093998724011749582473003682871994848593450, 10],
389: [61737418306809996908630595437832052272700263892021361415640028314169193468679,
38812622012907358702971098652989910931710935180589252493039839244464470874161, 12],
653: [96946680343613920300091880607027973891460464096741803273170797219756170839615,
8284378616542571170972530821008092883969646246950072002603300248219042690214, 9],
823: [101591078169875595753109991494905506967978136240961172388509275621325253165256,
10303457253039461887297603877250453486351456285884301634668455036581356870325, 1],
1151: [4942947285962241518079147671001480777229821084370946279070977308149340420785,
16048516228456466259745940658231903287363041924322695524263376011778714624389, 7],
1229: [91709248688928381574970306879959143256779911355970659117798234761550615769703,
85856020107303390216619790339131383987393998107943546966635616939179964220668, 1],
2447: [107091037109612570995136294213336682923913717986054179094643922074841981090569,
38297847735446351346601186761335949464902974429727652825128988635682228100545, 5],
4003: [69634612360547639692978050736475584000001950346963254134893659331303767659709,
7267690154676021708711188497795027916155791784399213931355851351510639163175, 6],
7103: [110323527740892356276833844768860449554291010208201255792825053403232044044793,
34702628194678317337541067016532288256370093140368313590333192034888987762792, 7],
7489: [69497610789705595174058737106242513100950130190920702467431032172354669590563,
43000989776377667520933328800675765150040604546037676698173382008099239610730, 1],
13003: [69994388431307856080322572731970917270151067511018517619530568914812259046195,
51645889020375608054366335957352074257130320976341249224010343992676177045239, 4],
16033: [80150849770701280770379260802876332257245651607220436873841708569583336291111,
67454443034144602807761039494179913732280388550455172486599878108995578176702, 6],
19423: [86763316696116146207846209443089376095966542281990071872698734124275764832625,
1497373281188841342082112917519408391664673991593710756225816313602284346637, 11],
30203: [27140306769124364253212826889951250714782929180685455599284687702513066987645,
27465668374540052358785326933904597047991904030378513297677516281690992773738, 1],
52183: [20786893006200668135980517481305198967871522130773700571327256180224225598537,
38476450712159672989047119873673988596095516096648184067210103163599625447149, 12],
72337: [16864673136043278693040185572303485743677125999233419976437302471094264721938,
39982110747848740588957884598316010802865483814669576940304708444368796141014, 9],
81173: [41965847134675666863089670621412699297207446259915277832939899605119013001686,
97595102592346869875749873612676528534971198259624427507680148771098555985918, 8]
}

```

Attack procedure

1. For each b' and G' of order o'
2. Send $B = -G'$ and $c = G'$ we will get back x coordinate of $G' * (d_a + 1)$
| Note that if we send 0, it will just return b' so no use
3. Lift the x coordinate **Over C'** to get $G' * (d_a + 1)$ But this wont help us to figure out of the two possible points. For this
4. Send another $B = -G'$ and $c = G' * 2$ to get back x coordinate of $G' * (d_a + 2)$
To figure out the correct lifting of the x coordinate to get $G' * (d_a + 1)$
5. Bruteforce and recover the ECDLP to get $d'_a = (d_a + 1) \bmod o'$
6. Combine all d'_a using chinese remainder theorem to get back original d_a
7. PROFIT ????

```
recovered_order = {}
for order, (gen_x, gen_y, b_i) in invalid_curves.items():
    gen = ECPoint(curve, gen_x, gen_y)
    bal, BG = get_decryption(-gen, gen) # gen*(da+1)
    BG_int = int.from_bytes(BG)
    y = modular_sqrt(BG_int**3+a*BG_int+b_i,p)
    point_BG = ECPoint(curve, BG_int, y)

    bal, BG2 = get_decryption(-gen, gen*2) # gen*(da+2)
    BG_int2 = int.from_bytes(BG2)
    if (point_BG+gen).x == BG_int2:
        recovered_order[order] = bruteforce(point_BG, gen, order)
    elif (point_BG-gen).x == BG_int2:
        recovered_order[order] = bruteforce(-point_BG, gen, order)
    else:
        print("something went wrong")

mods, values = [],[]
for i,v in recovered_order.items():
    mods.append(i)
    values.append((v-1)%i)

d_a = crt(mods,values)[0]

key_int = (d_a >> (8*16)) ^ (d_a & 0xffffffffffffffffffffffffffff)
key = key_int.to_bytes(16, 'big')
print(AES.new(key, AES.MODE_ECB).decrypt(flag2_enc))
```

ENO-be5t`th1nk`out`0f`th3`curv3"

Solve script

[solve.py](#)

| Interactive controls are omitted from this PDF. See the canonical HTML version.